

Exp no :10

Date :

## Use Ghidra to Disassemble and Analyze the Malware Code

### Project Objectives

- To provide a hands-on guide to disassembling and analyzing malware with Ghidra.
- To enable readers to recognize common malware functionalities, such as persistence mechanisms, anti-analysis techniques, and network communications.
- To equip users with the skills to interpret low-level assembly code and understand highlevel behaviors using Ghidra's decompiler.

### Prerequisites

- Ghidra: Installed and configured (requires Java Development Kit - JDK).
- Virtual Machine (VM): A secure, isolated, and virtualized environment (e.g., VMWare, VirtualBox) to safely handle binaries.
- Sample Binary: A benign sample or a controlled, safe hex dump for analysis (due to ethical/safety concerns).

### Project Outline (GitHub Repository Components)

A typical project structure for this analysis exercise would include the following components:

#### 1. Repository Root

| File/Folder | Description                                                                                    |
|-------------|------------------------------------------------------------------------------------------------|
| README.md   | Provides an overview of Ghidra, project requirements, and an outline of the analysis tutorial. |
| templates/  | Contains the template for documenting the forensic analysis findings.                          |
| docs/       | Contains the step-by-step analysis tutorials.                                                  |
| scripts/    | Stores automation scripts for Ghidra (Python/Java).                                            |
| samples/    | Stores the benign sample binaries or hex dumps for practice.                                   |

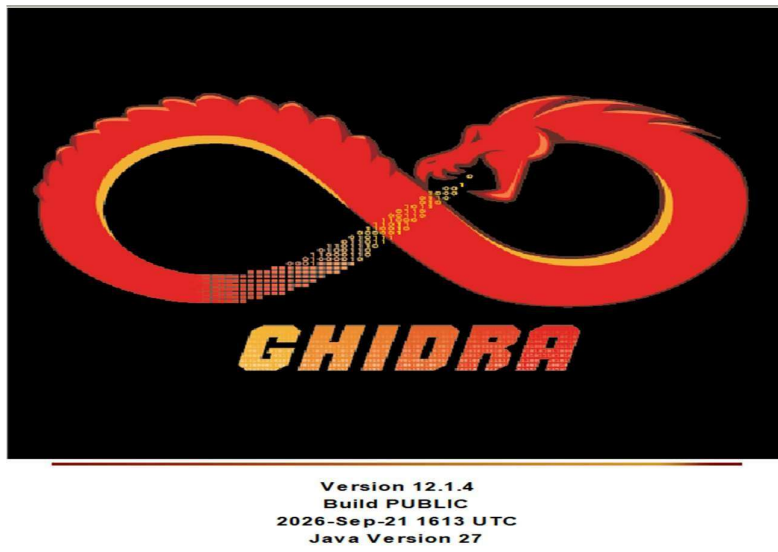
---

#### 2. Tutorial Steps (docs/Tutorial\_StepX.md)

Step-by-step instructions on performing static analysis using Ghidra:

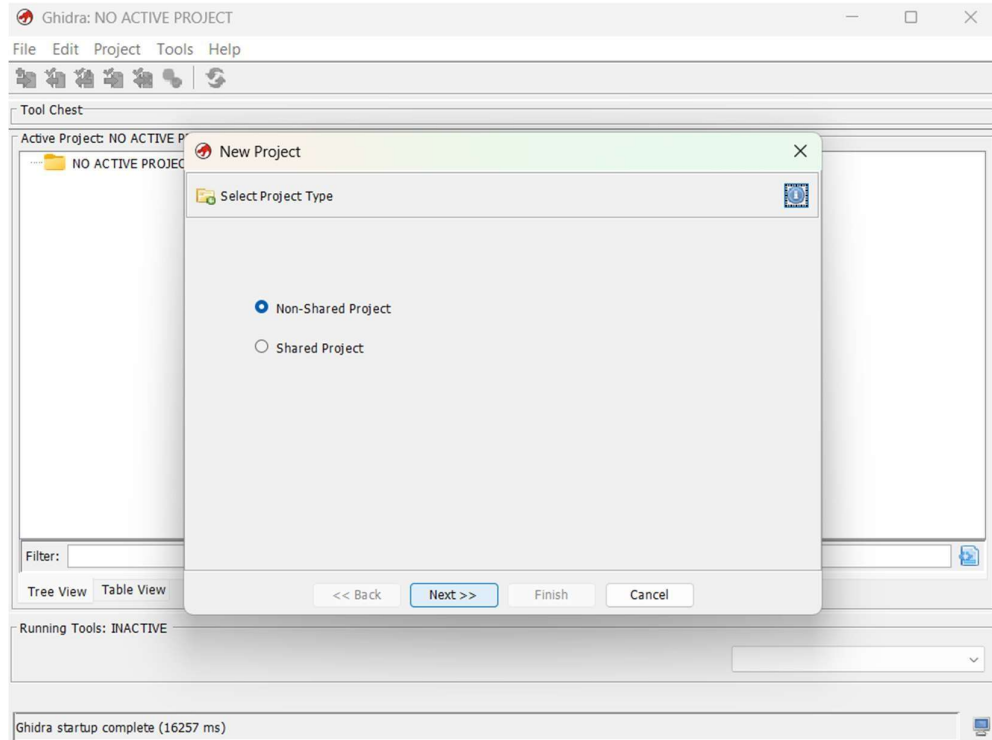
##### A. Environment Setup

- Instructions for setting up a virtualized and isolated analysis environment.
- Guide on installing Ghidra and verifying the Java dependency.



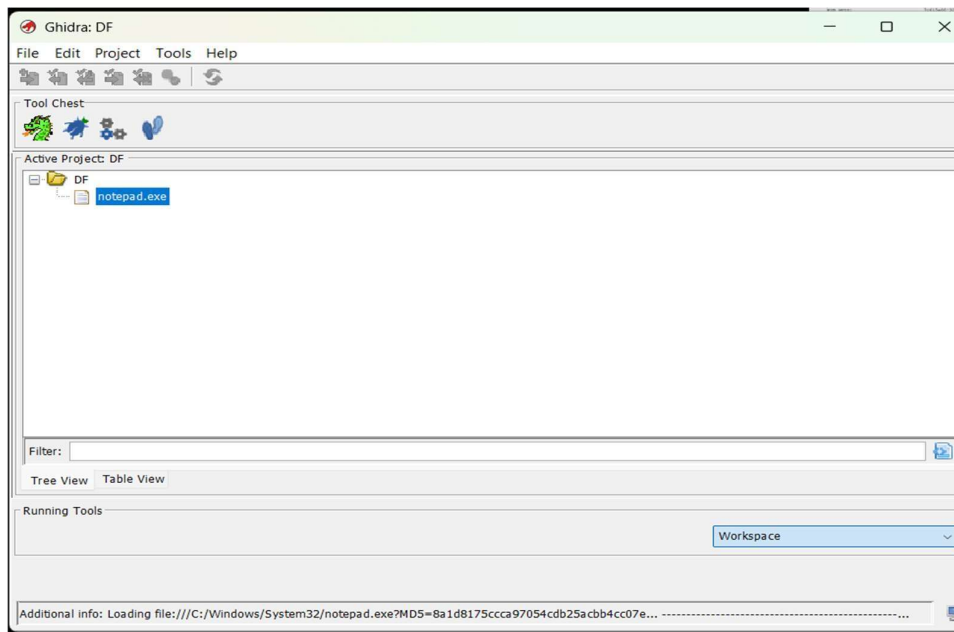
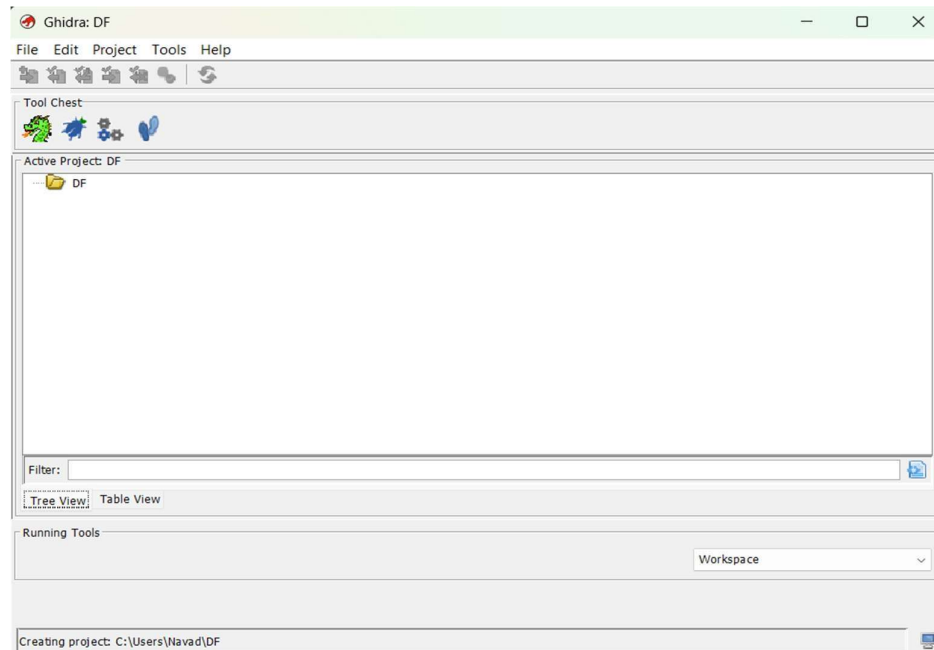
## B. Initial Analysis

- Loading the Binary: How to launch Ghidra, create a new project, and import the sample binary.
- Auto-Analysis: How to run the initial Ghidra auto-analysis and select appropriate analyzers.
- Identifying Entry Points: Locating the main function or the binary's execution starting point.



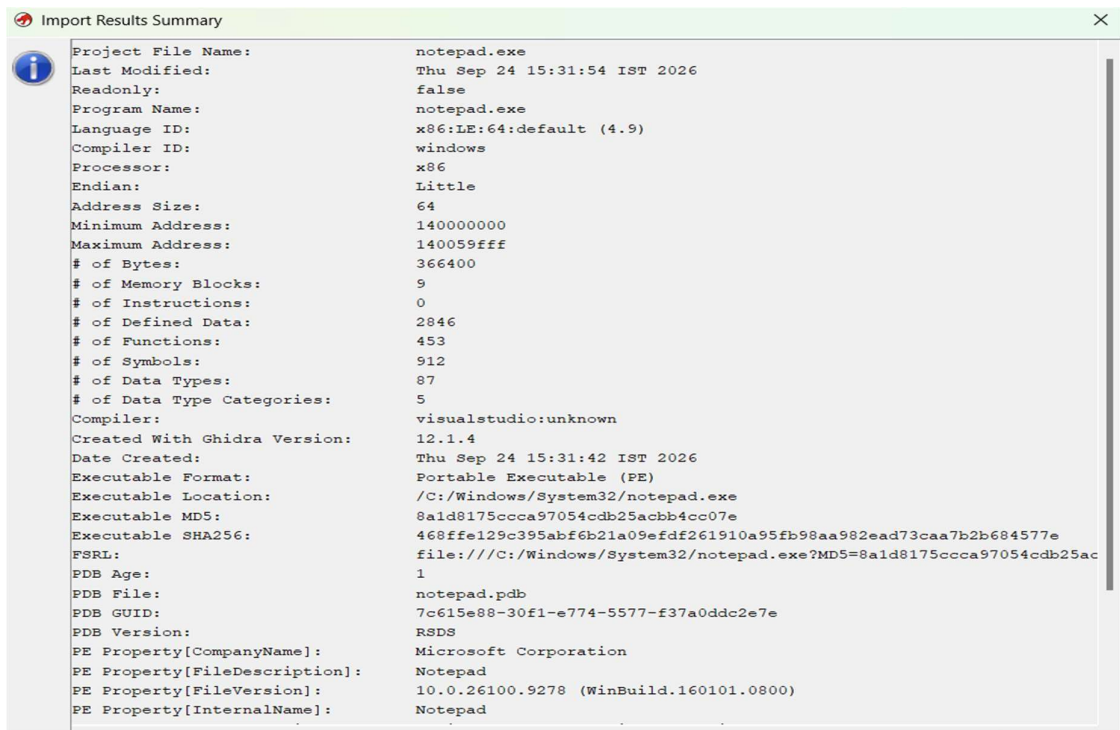
### C. Function Analysis

- **Decompilation:** Using Ghidra's Decompiler window to translate assembly into highlevel C-like code.
- **Function Identification:** Methods for renaming and analyzing critical functions to understand their purpose.
- **Cross-Referencing (XREF):** Utilizing Ghidra's cross-referencing features to trace where a function is called or where a variable is used.



#### D. String and Import Analysis

- Locating Strings: Using the Defined Strings window to find meaningful static strings (e.g., URLs, file paths, registry keys) that indicate malicious behavior.
- Interpreting Imports: Analyzing the Import Table to identify relevant imported functions (e.g., CreateFileA, URLDownloadToFile, RegSetValueEx) that suggest malware functionality.



- Introduction to using Ghidra's Control Flow Graphs (CFGs) for visualizing program execution flow.
- Examples of writing and running custom Ghidra scripts (Python/Java) for automation.

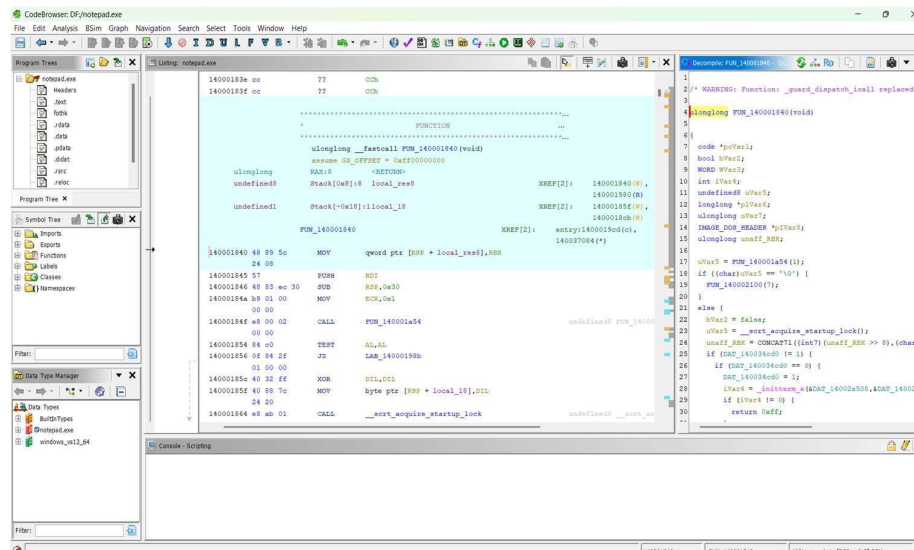
### 3. Example Automation Scripts (scripts/)

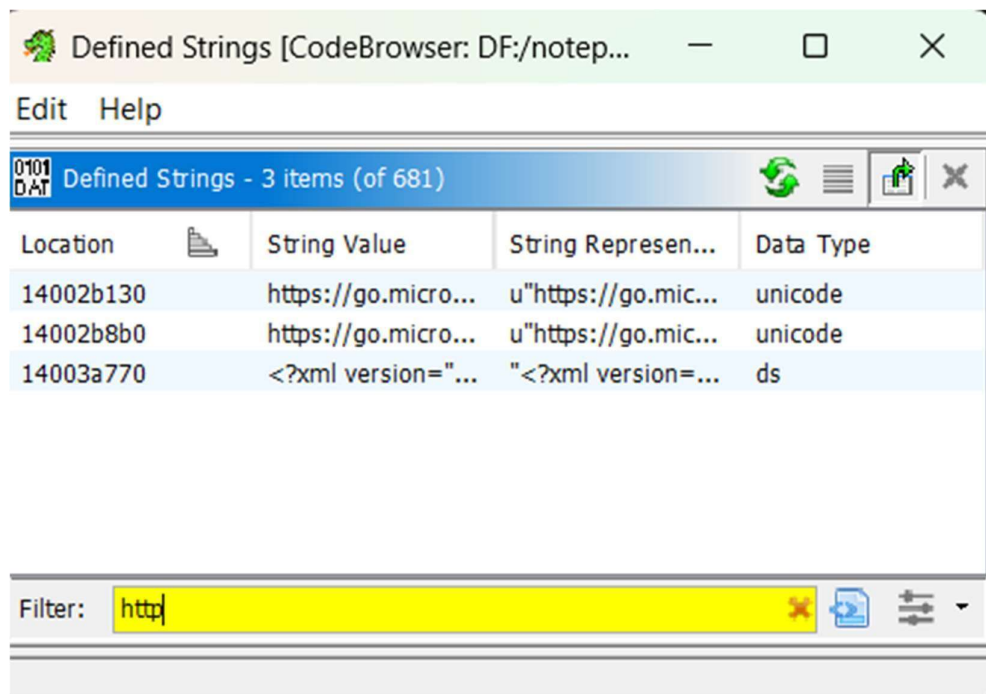
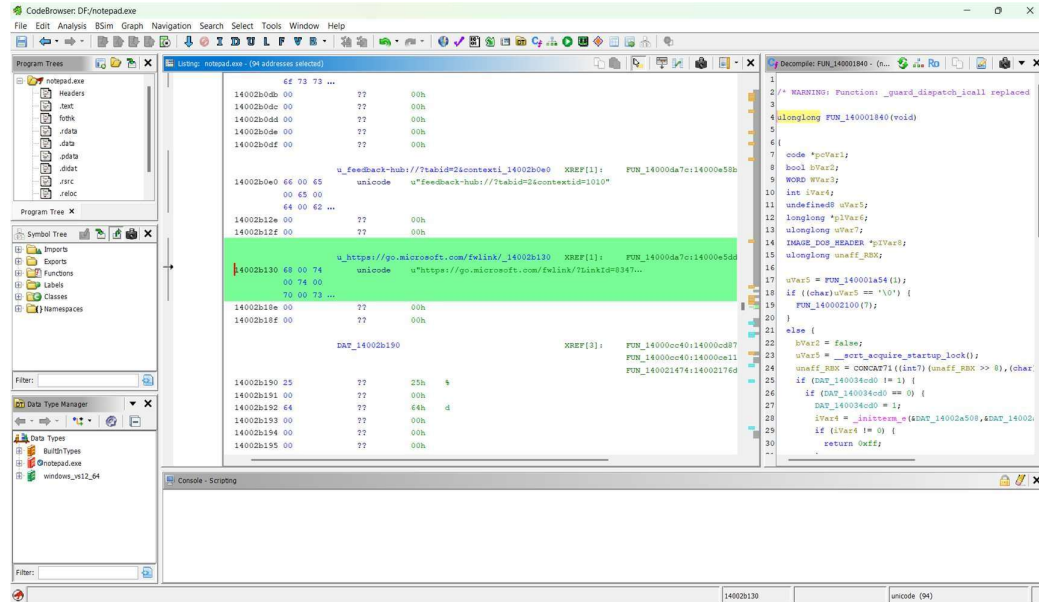
### Scripts using Ghidra's API to automate repetitive analysis tasks:

| Script Example                   | Purpose                                                                                                                                        |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>extract_strings.py</code>  | Automates the extraction and listing of all unique string references in the binary.                                                            |
| <code>network_analysis.py</code> | Identifies and flags functions that contain network-related API calls (e.g., <code>socket</code> , <code>connect</code> , <code>send</code> ). |
| <code>label_functions.py</code>  | Automates the labeling of functions and data segments based on known malicious patterns (e.g., file system manipulation, registry edits).      |

#### 4. Sample Data (samples/)

- **Benign Binary or Hex Dump:** Provide safe files for users to practice the disassembly steps. Example: `benign_sample1.bin` (A simple C program that mimics system calls or file writing but contains no harmful payload).





## 5. Report Template

A template to structure and document the findings of the Ghidra analysis:

1. Summary of Analysis: Brief description of the suspected malware and the scope of the analysis.
2. Function Analysis: Detailed summary of key functions, their purpose, and forensic findings.

3. Behavioral Indicators: Summary of observed behaviors (e.g., persistence, communication protocols, anti-analysis checks).
4. Recommendations: Suggestions for containment, eradication, or further analysis (e.g., dynamic analysis).

|

#### Rubrics

| Criteria & Marks Assigned                              | Mark Allotted | Mark Awarded |
|--------------------------------------------------------|---------------|--------------|
| 1. GitHub Activity & Submission Regularity             | 3             |              |
| 2. Application of Forensic Tools & Practical Execution | 3             |              |
| 3. Documentation & Reporting                           | 2             |              |
| 4. Engagement, Problem-Solving & Team Collaboration    | 2             |              |
| Total                                                  | 10            |              |

#### Result:

The experiment on malware analysis using Ghidra was successfully performed. The malware binary was disassembled, decompiled, and analyzed to identify its functions, strings, and behavioral patterns. This helped in understanding the internal logic, persistence mechanisms, and potential malicious activities of the code.